实验7 系统缺页次数统计

- 实验目的
  - 理解内存管理中缺页的概念
  - 综合运用实验1， 实验5， 实验6中/proc文件系统、内存管理、系统调用、内核编译的知识
  - 掌握向/proc文件系统中增加文件的方法
  - 掌握Linux内核模块的概念和操作方法

实验内容与要求

通过在内核中自建变量并利用/proc文件系统作为中介的方法来统计系统缺页次数

Linux内核模块

内核模块是Linux内核向外部提供的一个插口，简称模块。

Linux内核本身是一个单内核（monolithic kernel）。单内核的最大优点是效率高，因为所有的内容都集成在一起，但其缺点是可扩展性和可维护性相对较差，因此引入了模块机制弥补这一缺陷。

简言之，模块是一个为内核（从某种意义上来说，内核也是一个模块）或其他内核模块提供使用功能的代码块。可以被单独编译，但不能独立运行。它在运行时被链接到内核作为内核的一部分在内核空间运行，这与运行在用户空间的进程是不同的。模块通常由一组函数和数据结构组成，用来实现一种文件系统、一个驱动程序或其他内核上层的功能。

Linux内核模块

优点：

- 将内核映象的尺寸保持在最小，并具有最大的灵活性；
- 便于检验新的内核代码，而不需重新编译内核并重新引导。

缺点：

- 对系统性能和内存利用有负面影响；
- 装入的内核模块和其他内核部分一样，具有相同的访问权限，因此，差的内核模块会导致系统崩溃；
- 有些模块要求利用其他模块的功能，因此，内核要维护模块之间的依赖性。
- 内核版本和模块版本的不兼容，也可能导致系统崩溃。

构建内核模块

1. 编写模块代码，并以决定在哪里管理模块代码
   - 把模块代码加入到内核源代码树
   - 在内核源代码之外维护和构建内核源代码（本实验）
2. 编译运行
   - 保存代码，后缀名为xxx.c
   - 编写Makefile(注意M大写)
3. 加载模块
   - insmod xxx.ko

示例

```
#include <linux/init.h>
#include <linux/module.h>
MODULE_LICENSE("Dual BSD/GPL");

static int hello_init(void)
{
    printk(KERN_ALERT "Hello, World\n");
    return 0;
}

static void hello_exit(void)
{
    printk(KERN_ALERT "Goodbye ,cruel world\n");
}

module_init(hello_init);
module_exit(hello_exit);</span>
```

示例代码解读

常用头文件

module.h　　所有模块都要使用此头文件
kernel.h　　包含了常用的内核函数
init.h　　包含了宏_init和_exit，使系统在初始化完成后释放该函数，并释放其所占内存

必需模块函数

加载函数`module_init(hello_init)`
　　告诉内核你编写模块程序从那里开始执行
卸载函数`module_exit(hello_exit)`
　　告诉内核你编写模块程序从那里离开

示例代码解读

可选模块函数

MODULE_LICENSE("*******");	许可证申明
MODULE_AUTHOR("********");	作者申明
MODELE_DESCRIPTION("***");	模块描述
MODULE_VERSION("V1.0");	模块版本
MODULE_ALIAS("*********");	模块别名

编译运行模块代码

1. 写一个Makefile文件

- Makefile文件所做的工作是编译生成helloworld.o和helloworld.ko等文件
- hello.c只需在Makefile种添加一条指令：obj-m := hello.o2

2. 构建内核模块指令：

- make –C /kernelSrclocation SUBDIRS=$PWD modules

注：

1. Makefile 是一种脚本，这种脚本主要是用于多文件的编译
2. make 程序可以维护具有相互依赖性的源文件，但某些文件发生改变时，它能自动识别出，并只对相应文件进行自动编译。

加载模块

加载内核模块：

●载入模块最简单的方法是使用i命令：#insmod hello.ko

卸载内核模块：

●卸载模块使用命令：#rmmod hello.ko

注：

a) .ko 是kernel object 的缩写，是Linux 2.6内核使用的动态连接文件，在Linux系统启动时加载内核模块。

b) .o 是相当于windows中的.obj文件。.ko与.o的区别在于，.ko是linux 2.6内核编译之后生成的，多了一些module信息，如author，license之类的。.o文件则是linux 2.4内核编译生成的。

缺页次数

虚拟内存技术的应用使得进程线性地址空间里的页面不必常驻内存。当CPU请求一个不在内存中的页面时,会发生缺页,比如我们从内存读取/写入数据,而数据未在内存,此时都会发生缺页。缺页被定义为一种异常（缺页异常），会触发缺页中断处理流程

CPU结构提供一个do_page_fault函数处理缺页中断。由于每发生一次缺页都要进入缺页中断服务函数do_page_fault一次，所以统计该函数被调用的次数就可以得到系统从开机到现在的缺页次数

/proc文件系统简介

- 普通进程运行在用户态下，它们无法直接访问内核数据来了解系统信息。

- /proc虚拟文件系统，通过它里面的一些文件，可以获取系统状态信息并且修改某些系统的配置信息。
- /proc文件系统本身并不占用硬盘空间，它仅存在于内存之中，为操作系统本身和应用程序之间的通信提供了一个安全的界面。

- 可以简单地用cat，strings程序来查看这些信息。例如，查看系统内存的使用状况可以用如下的命令：
- #cat /proc/meminfo

/proc文件系统简介

/proc根下文件和目录

文件/目录名	描述
apm	高级电源管理信息
cmdline	内核命令行
cpuinfo	关于cpu信息
devices	可以用到的设备（块设备/字符设备）
dma	正在使用的DMA通道
filesystems	支持的文件系统
interrupts	中断的使用
meminfo	内存信息

/proc文件系统的文件记录了当前所有的系统信息，包括进程、文件系统、硬件等等。因此，可以通过在添加一个文件的方式，查看内存进程中的一些自定义运行参数，从而达到使用/proc实现内核与用户空间通信的目的。

/proc文件创建和删除

所需头文件	#include <linux/proc_fs.h>
创建目录	struct proc_dir_entry* proc_mkdir (const char *name,struct proc_dir_entry *parent); @name :要创建的目录名 @parent :这个目录的父目录
创建只读文件	struct proc_dir_entry* create_proc_read_entry (const char *name,mode_t mode,struct proc_dir_entry *parent,read_proc_t* read_proc,void *data); @name :要创建的文件名 @mode :要创建的文件的属性 默认0755 @parent :这个文件的父目录 @read_proc :当用户读这个文件时，内核调用的函数 @data :传给read_proc的参数

/proc文件创建和删除（续）

所需头文件	#include <linux/proc_fs.h>
删除文件和目录	void remove_proc_entry (const char *name,struct proc_dir_entry *parent); @name :要删除的文件或目录名 @parent :所在的父目录
只读回调函数声明	typedef int (read_proc_t)(char *page, char **start, off_t off,int count, int *eof, void *data);

实验内容与要求

通过在内核中自建变量并利用/proc文件系统作为中介的方法来统计系统缺页次数

(1)在内核中实现缺页次数统计

(2)编译并安装新内核

(3)新建内核模块，并加载到新内核，通过/proc实现用户态下查看缺页次数

实验步骤

第一部分：在内核中实现缺页次数统计

```
ubuntu@oslinux-virtual-machine:~$
cd /usr/src/linux-2.6.32.60
//切换到预编译内核目录
ubuntu@oslinux-virtual-machine:~$ /usr/src/linux-2.6.32.60$
 sudo gedit arch/x86/mm/fault.c
//在do_page_fault函数上一行定义统计缺页次数全局变量pfcount
unsigned long volatile pfcount;
//将pfcount加入到do_page_fault中，用以统计缺页次数.
pfcount++;
```

```
unsigned long volatile pfcount;
/*
 * This routine handles page faults.  It determines the address,
 * and the problem, and then passes it off to one of the appropriate
 * routines.
 */
dotraplinkage void __kprobes
do_page_fault(struct pt_regs *regs, unsigned long error_code)
{
        struct vm_area_struct *vma;
        struct task_struct *tsk;
        unsigned long address;
        struct mm_struct *mm;
        int write;
        int fault;
        pfcount++;
```

实验步骤

ubuntu@oslinux-virtual-machine:~$ /usr/src/linux-2.6.32.60$ sudo gedit include/linux/mm.h

//声明全局变量pfcount到头文件mm.h中，在extern int page_cluster下面添加代码：

extern unsigned long volatile pfcount;

```
extern unsigned long num_physpages;
extern unsigned long totalram_pages;
extern void * high_memory;
extern int page_cluster;
extern unsigned long volatile pfcount;

#ifdef CONFIG_SYSCTL
extern int sysctl_legacy_va_layout;
```

实验步骤

```
ubuntu@oslinux-virtual-machine:~$ /usr/src/linux-2.6.32.60$
 sudo gedit kernel/kallsyms.c
//导出pfcount全局变量，让整个内核都可以访问



//最后一行添加代码如下：
EXPORT_SYMBOL(pfcount);
```

下面开始编译内核，和系统调用实验编译内核步骤一样。

实验步骤

第二部分：编译安装新内核

```
ubuntu@oslinux-virtual-machine:~$ /usr/src/linux-2.6.32.60$
sudo make mrproper
//删除原来编译产生的垃圾
ubuntu@oslinux-virtual-machine:~$ /usr/src/linux-2.6.32.60$
sudo cp /boot/config-3.2.0-29-generic-pae  .config
//导入原系统的内核配置
```

实验步骤

ubuntu@oslinux-virtual-machine:~$ /usr/src/linux-2.6.32.60$

sudo make oldconfig

//更改配置，一直回车

（注：对于没有实验6没有安装成功的同学，可尝试使用sudo make menuconfig命令代替上述sudo make oldconfig 命令

// Load an Alternate Configuration File -> 导入.config文件 -> Save an Alternate Configuration File -> Exit

//导入配置到欲编译内核中 ）

ubuntu@oslinux-virtual-machine:~$ /usr/src/linux-2.6.32.60$ sudo make clean

实验步骤

```
ubuntu@oslinux-virtual-machine:~$ /usr/src/linux-2.6.32.60$
sudo make bzImage//编译内核
ubuntu@oslinux-virtual-machine:~$ /usr/src/linux-2.6.32.60$
sudo make modules//编译模块
ubuntu@oslinux-virtual-machine:~$ /usr/src/linux-2.6.32.60$
sudo make modules_install//添加模块到内核
ubuntu@oslinux-virtual-machine:~$ /usr/src/linux-2.6.32.60$
sudo make install//链接、组建内核
ubuntu@oslinux-virtual-machine:~$ /usr/src/linux-2.6.32.60$
sudo reboot//重新启动Linux系统
```

实验步骤

重新启动Linux系统，开机时长按shift键直到进入启动加载界面，选择新编译的内核版本，按回车进入系统。进入系统后，使用命令uname -a 验证内核版本是否安装成功。

```
GNU GRUB  version 1.99-21ubuntu3.1

Ubuntu, with Linux 3.2.0-29-generic-pae
Ubuntu, with Linux 3.2.0-29-generic-pae (recovery mode)
Previous Linux versions
Memory test (memtest86+)
Memory test (memtest86+, serial console 115200)
```

```
GNU GRUB  version 1.99-21ubuntu3.1

Ubuntu, with Linux 2.6.32.60
Ubuntu, with Linux 2.6.32.60 (recovery mode)
```

```
oslinux@oslinux-virtual-machine:~$ uname -a
Linux oslinux-virtual-machine 2.6.32.60 #1 SMP Sat Oct 22 23:43:17 CST 2016 i686
 athlon i386 GNU/Linux
oslinux@oslinux-virtual-machine:~$
```

实验步骤

第三部分：新建内核模块，通过/proc实现用户态下查看缺页次数

```
ubuntu@oslinux-virtual-machine:~$ mkdir source
//系统重启后，创建source文件夹，用于存放编写的模块程序
ubuntu@oslinux-virtual-machine:~$ cd source/
//切换到source目录
ubuntu@oslinux-virtual-machine:~/source$ gedit pf.c
//新建用于构建模块的代码
```

实验步骤

```
#include <linux/mm.h>
#include <linux/proc_fs.h>
#include <linux/string.h>
#include <asm/uaccess.h>

struct proc_dir_entry *proc_pf;
struct proc_dir_entry *proc_pfcount;
extern unsigned long volatile pfcount;
static inline struct proc_dir_entry *proc_pf_create(const char* name, mode_t mode, read_proc_t *get_info)
{
    return create_proc_read_entry(name, mode, proc_pf, get_info, NULL);
}

int get_pfcount(char *buffer, char **start, off_t offset, int length, int *peof, void *data)
{
    int len = 0;
    len = sprintf(buffer, "%ld \n", pfcount);
    return len;
}
static int pf_init(void)
{
    proc_pf = proc_mkdir("pf", 0);
    proc_pf_create("pfcount", 0, get_pfcount);
    return 0;
}

static void pf_exit(void)
{
    remove_proc_entry("pfcount", proc_pf);
    remove_proc_entry("pf", 0);
}

module_init(pf_init);
module_exit(pf_exit);
```

/proc/pf/pfcount

实验步骤

```
/*pf.c*/
/*内核模块代码*/
#include <linux/init.h>
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/mm.h>
#include <linux/proc_fs.h>
#include <linux/string.h>
#include <asm/uaccess.h>

struct proc_dir_entry *proc_pf;
struct proc_dir_entry *proc_pfcount;
extern unsigned long volatile pfcount;
```

实验步骤

```
static inline struct proc_dir_entry *proc_pf_create(const
char* name,
  mode_t mode, read_proc_t *get_info)
{
    return create_proc_read_entry(name, mode, proc_pf,
get_info, NULL);
}

int get_pfcount(char *buffer, char **start, off_t offset, int
length, int *peof,void *data)
{
    int len = 0;
    len = sprintf(buffer, "%ld \n", pfcount);
    return len;
}
```

实验步骤

```
static int pf_init(void)
{
    proc_pf = proc_mkdir("pf", 0);
    proc_pf_create("pfcount", 0, get_pfcount);
    return 0;
}

static void pf_exit(void)
{
    remove_proc_entry("pfcount", proc_pf);
    remove_proc_entry("pf", 0);
}

module_init(pf_init);
module_exit(pf_exit);
MODULE_LICENSE("GPL");
MODULE_AUTHOR("Aron.t.wang");
```

实验步骤

```
ubuntu@oslinux-virtual-machine:~/source$
gedit Makefile
//构建Makefile，在Makefile中添加如下内容：
obj-m := pf.o
ubuntu@oslinux-virtual-machine:~/source$
sudo make -C  /usr/src/linux-2.6.32.60 SUBDIRS=$PWD modules
//编译、构建内核模块
ubuntu@oslinux-virtual-machine:~/source$
sudo insmod pf.ko
//加载内核模块到内核中
```

实验步骤

ubuntu@oslinux-virtual-machine:~/source$ sudo lsmod | grep pf

//验证pf内核模块是否加载到内核进程中，运行命令出现pf信息说明加载成功

```
oslinux@oslinux-virtual-machine:~/source$ sudo lsmod | grep pf
pf                      798  0
```

ubuntu@oslinux-virtual-machine:~/source$ cat /proc/pf/pfcount

//打印/proc/pf/pfcount，查看缺页统计次数

```
oslinux@oslinux-virtual-machine:~/source$ cat /proc/pf/pfcount
1034116
```